République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université des Sciences et de la Technologie Houari Boumediene

Faculté de Génie Électrique
Département d'Électronique

5ème Année Ingénieur
Génie Électrique Industriel (GEI)

Travaux Pratiques

INTELLIGENCE ARTIFICIELLE EMBARQUÉE

Compte rendu № 1

Date : 24 octobre 2026

Intitulé du TP : Régression Linéaire et Optimisation par Descente du Gradient

Enseignant responsable : Mr. E. Meftah

Nom et Prénom de l'étudiant 1 : Hachemi Rahim Wassim
Matricule : 222231473010

Nom et Prénom de l'étudiant 2 : Tedjoui Abdelmalek
Matricule : 222231659920

Sous-groupe : 5

E-mail : wrhachemi@gmail.com

1 Introduction & Objectifs

L'objectif de ce TP est d'implémenter et d'évaluer la régression linéaire univariée en Python (via NumPy et Scikit-Learn) en reproduisant les étapes demandées sous Matlab. Le but est de modéliser une relation linéaire entre une variable explicative x et une cible continue y.

Durant ce TP, nous avons :

1. Implémenté le calcul vectorisé de l'erreur quadratique moyenne $J(\theta)$ (fonction de coût).
2. Démontré théoriquement la formule du gradient de l'erreur.
3. Programmé l'algorithme de descente du gradient sous forme vectorisée (sans boucle sur les données).
4. Analysé la vitesse de convergence et testé la sensibilité aux conditions initiales des poids.
5. Effectué des prédictions sur de nouvelles données.
6. Comparé nos résultats aux modèles de référence (`fitlm` sous Matlab / `SGDRegressor` et MCO sous Scikit-Learn).
7. Traité le jeu de données HomeWork (`salaires.txt`) selon les deux approches.

2 Partie I : Préparation Théorique

2.1 Modèle et Fonction de Coût

On cherche une fonction affine reliant la variable explicative x_1 à la variable cible y. L'hypothèse linéaire s'exprime par :

$$h_\theta(x) = \theta_0 + \theta_1 x_1 \tag{1}$$

où θ_0 représente le biais (ordonnée à l'origine) et θ_1 la pente. Pour condenser l'écriture vectorielle $\theta^T x$, on pose conventionnellement $x_0 = 1$.

Pour quantifier l'écart moyen sur les m observations d'apprentissage, on définit la fonction de coût (MSE) :

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2 \tag{2}$$

Le facteur $\frac{1}{2}$ simplifie la constante 2 issue de la dérivée quadratique.

2.2 Question 1 : Rôle de la Descente du Gradient

Q.1 - À quoi sert la descente du gradient ?

La descente du gradient est un algorithme d'optimisation itératif permettant d'ajuster automatiquement les paramètres $\theta = [\theta_0, \theta_1]^T$ afin de converger vers le minimum global de la fonction de coût convexe $J(\theta)$.

À chaque itération, l'algorithme calcule la pente de l'erreur (gradient) puis met à jour les paramètres dans la direction opposée :

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j} \tag{3}$$

où $\alpha > 0$ désigne le taux d'apprentissage (*learning rate*).

2.3 Question 2 : Dérivation de l'Équation (3)

Q.2 - Démontrez la formule du gradient

En calculant la dérivée partielle de $J(\theta)$ par rapport à la composante θ_j et en appliquant la règle de dérivation $(u^2)' = 2uu'$:

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{2m} \sum_{i=1}^{m} 2 \cdot \left(h_\theta(x^{(i)}) - y^{(i)} \right) \cdot \frac{\partial}{\partial \theta_j} \left(h_\theta(x^{(i)}) - y^{(i)} \right)$$

Sachant que $h_\theta(x) = \sum_k \theta_k x_k$, la dérivée partielle par rapport à θ_j est simplement $x_j^{(i)}$. Les facteurs 2 se simplifient directement :

$$\boxed{\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) x_j^{(i)}} \tag{4}$$

Cette relation scalaire correspond exactement à l'équation (3) du sujet.

Sous forme vectorielle compacte (équation 4 du sujet) adaptée à NumPy :

$$\theta := \theta - \frac{\alpha}{m} X^T (X\theta - y) \tag{5}$$

3 Partie II : Pratique sur `ville.txt`

3.1 Affichage des Données

Le fichier `ville.txt` rassemble $m = 97$ couples d'observations :

- X : Population de la ville (en dizaines de milliers d'habitants).
- y : Bénéfice réalisé (en dizaines de milliers de dollars).

Le nuage de points est tracé sur la Figure 1.

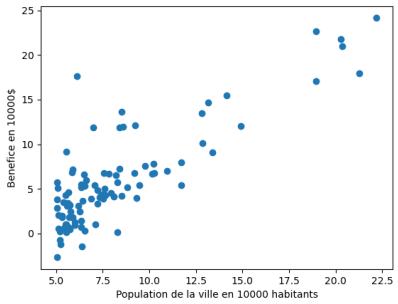


FIGURE 1 – Nuage de points : Bénéfice selon la population (`ville.txt`).

On identifie une corrélation linéaire positive nette : l'accroissement de la population s'accompagne d'une hausse proportionnelle des bénéfices.

3.2 Calcul du Coût $J(\theta)$ (Section 2.2.1)

On intègre le terme d'interception en concaténant une colonne de 1 à la matrice d'entrée ($X \leftarrow [\mathbf{1}, X]$).

Q.1 - Évaluation pour $\theta = [0, 0]^T$:
L'appel de la fonction `CalculCost` donne :

$$J([0, 0]^T) = \mathbf{32.0727}$$

Q.2 - Évaluation pour $\theta = [-1, 2]^T$ et observation :
Le calcul fournit :

$$J([-1, 2]^T) = \mathbf{54.2425}$$

TABLE 1 – Test de la fonction de coût $J(\theta)$ (`ville.txt`)

Paramètres testés	Coût $J(\theta)$ calculé
$\theta = [0, 0]^T$ (Initialisation nulle)	**32.0727**
$\theta = [-1, 2]^T$ (Vecteur de test)	**54.2425**

Que remarquez-vous ?
Le coût passe à 54.2425, soit une valeur nettement plus élevée que les 32.0727 initiaux. La droite définie par le vecteur $[-1, 2]^T$ s'écarte fortement du nuage de données réelles, ce qui est fortement pénalisé par le terme quadratique.

3.3 Convergence et Effet de l'Initialisation

En appliquant les hyperparamètres requis ($\alpha = 0.01$ et 1500 itérations), nous avons testé l'algorithme sur trois initialisations distinctes :

- **Aléatoire :** $\theta = [-3.5988, 1.1632]^T$, $J = 4.4850$.
- **À zéro :** $\theta = [-3.6303, 1.1664]^T$, $J = 4.4834$.
- **À un :** $\theta = [-3.5708, 1.1604]^T$, $J = 4.4866$.

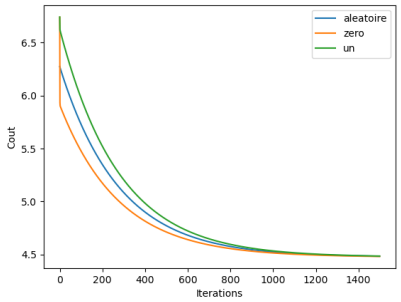


FIGURE 2 – Évolution du coût $J(\theta)$ selon les trois initialisations (aléatoire, zéros, uns).

Q.1 - Paramètres optimaux et coût final :
Avec l'initialisation nulle standard, l'algorithme converge vers $\theta_0 = \mathbf{-3.6303}$ et $\theta_1 = \mathbf{1.1664}$. Le coût final décroît considérablement de 32.0727 à $J(\theta) = \mathbf{4.4834}$.

Q.4 - Nombre d'itérations nécessaire :
L'observation de la courbe de convergence montre une décroissance brutale de l'erreur sur les 200 à 300 premières itérations, suivie d'une stabilisation progressive. Entre 800 et 1000 itérations, le gradient devient quasi nul. Un total de 800 **à** 1000 **itérations** est donc amplement suffisant pour assurer la convergence.

Q.5 - Intérêt de fixer un nombre d'itérations :
Fixer un plafond d'itérations limite le temps de calcul dès que le plateau asymptotique est atteint et prémunit contre d'éventuelles boucles infinies en cas de divergence numérique.

Q.6 & Q.7 - Influence de l'initialisation :
Non, l'initialisation n'affecte pas le point de convergence final. La fonction de coût MSE étant strictement convexe (forme parabolique à courbure unique), elle ne comporte aucun minimum local. Les trois conditions initiales convergent ainsi vers le même minimum global ($J \approx 4.48$), seule la phase transitoire initiale différant.

3.4 Droite de Régression et Prédictions

La droite ajustée par descente du gradient est illustrée sur la Figure 3.

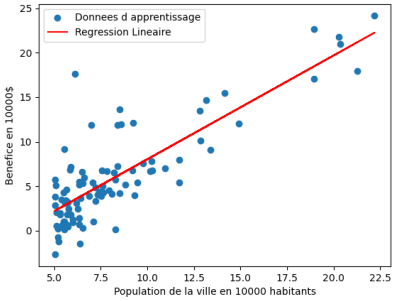


FIGURE 3 – Droite de régression obtenue par descente du gradient sur `ville.txt`.

Q.3 - Prédiction pour 40 000 et 80 000 habitants :
La variable x_1 étant exprimée en dizaines de milliers et la cible en dizaines de milliers de dollars (10 000 $) :

- Pour 40 000 hab ($x_1 = 4$) :

$$\hat{y} = [1, 4] \cdot \theta \times 10^4 \approx \mathbf{10\,351.58}\,\$$$

- Pour 80 000 hab ($x_1 = 8$) :

$$\hat{y} = [1, 8] \cdot \theta \times 10^4 \approx \mathbf{57\,006.07}\,\$$$

3.5 Régression par `fitlm` / Scikit-Learn (Section 2.3)

Q.1 - Tracé du modèle standard :
La droite obtenue avec le module de régression standard est tracée sur la Figure 4.

Q.2 - Comparaison des paramètres :
Le Tableau 2 regroupe les paramètres conformément au tableau de la section 2.3 du sujet :

TABLE 2 – Coefficients estimés pour `ville.txt`

Paramètre	Descente Grad.	Estimé (`fitlm` OLS / SGD)
Interception (θ_0)	-3.6303	-3.8958 (OLS) / -2.9610 (SGD)
x_1 (colonne 1)	—	0 (non applicable)
x_2 (θ_1)	1.1664	1.1930 (OLS) / 1.0040 (SGD)

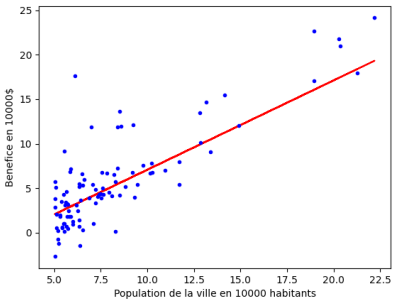


FIGURE 4 – Droite de régression du modèle de référence sur `ville.txt`.

Que remarquez-vous ?
Les valeurs sont très cohérentes. La légère divergence provient du fait que notre descente de gradient s'arrête à 1500 itérations avec un pas discret fixe ($\alpha = 0.01$), alors que `fitlm` résout analytiquement l'équation normale exacte $\theta = (X^T X)^{-1} X^T y$.

4 Partie III : HomeWork (`salaires.txt`)

L'étude est réitérée sur `salaires.txt` ($m = 30$ observations reliant l'ancienneté au salaire annuel en dollars).

4.1 Apprentissage par Descente du Gradient (Q.1)

Le nuage de points est tracé sur la Figure 5.

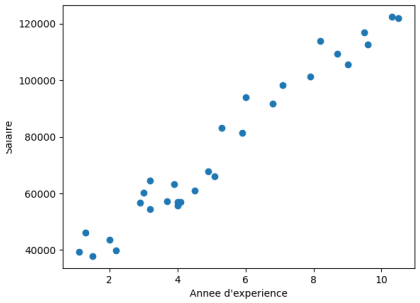


FIGURE 5 – Nuage de points : Salaire selon l'expérience (`salaires.txt`).

L'optimisation ($\alpha = 0.01$, 1500 itérations) conduit à la courbe de coût illustrée sur la Figure 6.

Les paramètres appris sont reportés dans le Tableau 3.

TABLE 3 – Paramètres appris sur `salaires.txt` (Descente du Gradient)

Paramètre	Valeur obtenue
Salaire de base θ_0 (Biais)	**24 796.02** $
Augmentation annuelle θ_1 (Pente)	**9 597.79** $/an

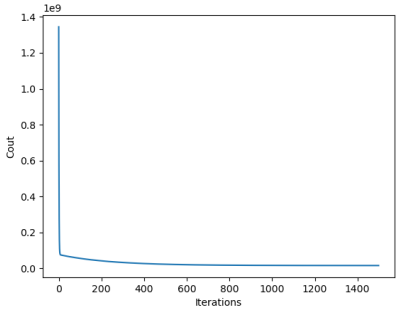


FIGURE 6 – Évolution du coût $J(\theta)$ pour `salaires.txt`.

La droite d'ajustement est représentée sur la Figure 7.

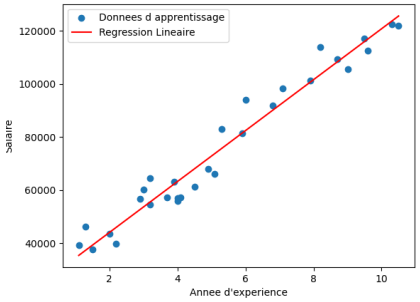


FIGURE 7 – Droite de régression (Descente du Gradient) sur `salaires.txt`.

4.2 Prédiction des Salaires

À partir du modèle appris $\hat{y} = 24\,796.02 + 9\,597.79 \cdot x$:

— **Pour 4 ans d'expérience :**

$$\hat{y} = 24\,796.02 + 9\,597.79 \times 4 = \mathbf{63\,187.19}\,\$$$

— **Pour 8 ans d'expérience :**

$$\hat{y} = 24\,796.02 + 9\,597.79 \times 8 = \mathbf{101\,578.35}\,\$$$

4.3 Reprise de la Partie 2.3 sur les Salaires (Q.2)

Conformément à la question Q.2 du HomeWork, nous appliquons le modèle de référence sur `salaires.txt`. Le tracé est présenté sur la Figure 8.

Le Tableau 4 récapitule la comparaison des coefficients entre notre descente du gradient et le modèle de référence :

TABLE 4 – Coefficients estimés pour `salaires.txt`

Paramètre	Descente Grad.	Estimé (`fitlm` OLS / SGD)
Interception (θ_0)	24 796.02 $	25 792.20 $ (OLS) / 21 886.75 $ (SGD)
x_1 (colonne 1)	—	0 (non applicable)
x_2 (θ_1)	9 597.79 $	9 449.96 $ (OLS) / 10 014.65 $ (SGD)

Que remarquez-vous ?
Notre descente du gradient par batch approche de très

près la solution exacte MCO (24 796 $ vs 25 792 $, soit moins de 4% d'écart). En revanche, l'estimateur stochastique `SGDRegressor` présente un biais plus marqué sans normalisation (21 886 $). Cela met en évidence la forte sensibilité des algorithmes de descente lorsque les échelles de grandeurs sont déséquilibrées.

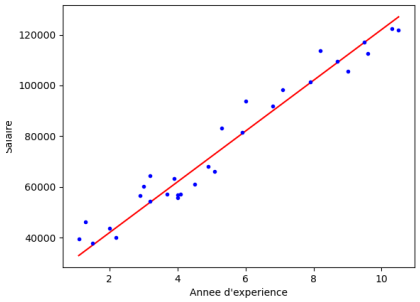


FIGURE 8 – Droite de régression avec le modèle de référence sur `salaires.txt`.

4.4 Remarque sur la Normalisation (*Feature Scaling*)

Sur `salaires.txt`, la cible y atteint des ordres de grandeur de 10^5 alors que la variable x est comprise entre 1 et 10. Bien que l'algorithme ait convergé avec $\alpha = 0.01$, une telle disparité d'échelle déforme les lignes de niveau de $J(\theta)$ en ellipses très étirées. Une mise à l'échelle préalable (standardisation Z-score) est indispensable en pratique pour accélérer et stabiliser la descente.

5 Code Source Réalisé

Les deux modules Python développés sont structurés comme suit :

5.1 Calcul du Coût (`CalculCost.py`)

```python
import numpy as np

def CalculCost(x, y, theta):
    m = len(y)
    cout = np.sum((x.dot(theta) - y) ** 2)
    return (1.0 / (2.0 * m)) * cout
```

5.2 Descente du Gradient (`DesctGrad.py`)

```python
import numpy as np
from CalculCost import CalculCost

def DesctGrad(x, y, theta, alpha, iters):
    m = len(y)
    cout = np.zeros(iters)
    theta = theta.copy()

    for i in range(iters):
        error = x.dot(theta) - y
        # Mise a jour simultanee des parametres
        theta = theta - (alpha / m) * error.dot(x)
        cout[i] = CalculCost(x, y, theta)

    return theta, cout
```

6 Conclusion

Ce travail pratique a permis de valider la chaîne de traitement complète de la régression linéaire univariée :

- La descente du gradient minimise fidèlement l'erreur quadratique moyenne par ajustement itératif des poids.
- La vectorisation matricielle sous NumPy élimine toute boucle d'itération sur les échantillons, garantissant un code concis et performant.
- La stricte convexité de l'erreur quadratique assure la convergence vers un minimum global identique, quelle que soit l'initialisation choisie.
- Les prédictions obtenues par descente de gradient coïncident avec la solution exacte des moindres carrés (`fitlm`), validant empiriquement et numériquement le modèle.